

Student ID:

Student Name:

Course: Data Structures (CSE CS203A)

Assignment V: Tree

Student Worksheet Companion

Due date: 2025.12.30 23:59:59

Section 1. Definitions of Tree Variants

Task: Write your own definitions for each tree type. You may use AI for learning, but rewrite in your own words.

1. General Tree

Definition: 樹狀結構中最基本的形式，由節點組成階層架構，沒有子節點的數量限制，可以是 0 或無限多

2. Binary Tree

Definition: 與 general tree 不同，每個子節點最多只能有兩個子節點，他們有特定的名稱和位置，分別是左子節點和右子節點

3. Complete Binary Tree

Definition: 是 binary tree 的分支，著重於填滿空間，絕不會有空缺，所以很省空間，必須滿足除了最後一層，每一層皆是填滿的，以及最後一層需靠左對齊，由上到下、由左至右

4. Binary Search Tree (BST)

Definition: 是有排序功能的二元樹，能快速搜尋，對於樹中每一個節點，左邊子樹的所有數值一定要小於該節點，而右邊子樹的所有數值一定要大於該節點

5. AVL Tree

Definition: 解決樹中不平衡導致時間複雜度增加的問題，依賴數學標準保持平衡，平衡因子 = 左子樹高度 - 右子樹高度，其平衡因子絕對值不能超過 1，當不平衡時執行旋轉來修復

6. Red-Black Tree

Definition: 在節點上儲存紅色或黑色的資訊，透過規則保證最長路徑不會超過最短路徑的兩倍，每個節點不是黑色就是紅色，根一定為黑色，所有底部的空節點都視為黑色，紅色節點的子節點必為黑色，且從一節點出發走到底下任一節點經過的黑色節點數量需依樣多

7. Max Heap

Definition: 是一顆 Complete Binary Tree，滿足父節點的值 $\geq$ 子節點的值，所以 root 一定為樹中的最大值，但不保證左邊比右邊大，或右邊比左邊大

8. Min Heap

Definition: 相反於 Max heap，Min tree 中所有父節點的值 $\leq$ 子節點的值，root 為樹中最小值

Section 2. Tree Family Hierarchy and Transformations

Task: Show how these structures are related (general $\rightarrow$ specialized). Use a simple diagram and explanations of what constraints are added at each step.

Student ID:

Student Name:

2.1 Tree Family Diagram

You may draw this by hand and paste a photo, or use drawing tools.

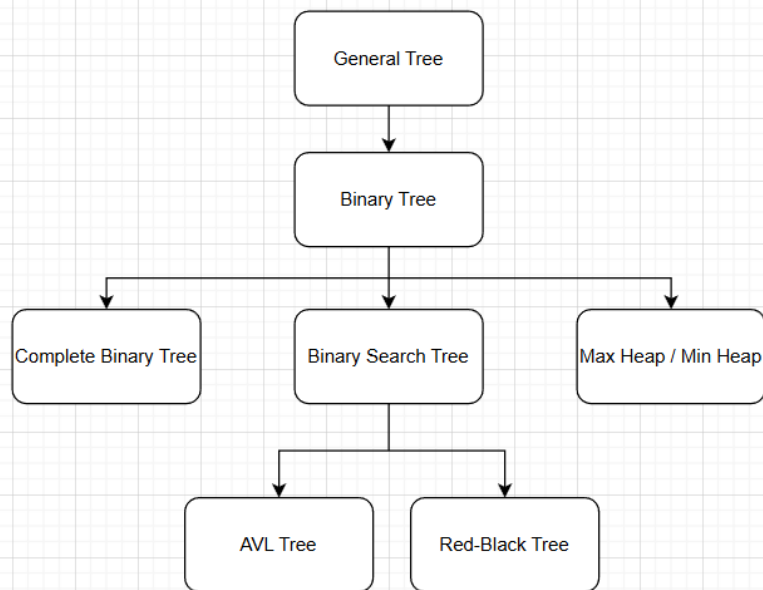
Suggested chain example (you may extend or adjust):

General Tree → Binary Tree → Complete Binary Tree

Binary Tree → Binary Search Tree → AVL / Red-Black

Binary Tree → Max Heap / Min Heap

Your Diagram:



<https://app.diagrams.net/?src=about>

2.2 Explanation of Transformations

Fill in what new property or constraint is added at each step.

From	To	New property / constraint added
General Tree	Binary Tree	每一節點最多只能有兩個節點
Binary Tree	Complete Binary Tree	所有節點須向左排滿
Binary Tree	Binary Search Tree	依據左小又大進行排序
BST	AVL Tree	進行層數的平衡，避免傾斜
BST	Red-Black Tree	增加紅黑顏色存入減少維護成本
Binary Tree	Max Heap	加入上下層間的大小關係，上需大於下
Binary Tree	Min Heap	加入上下層間的大小關係，上需小於下

Student ID:

Student Name:

Section 3. Tree Constructions Using Given Integers

Given integers (fixed for all parts):

37, 142, 5, 89, 63, 117, 24, 176, 58, 133, 92, 11, 151, 72, 39, 184, 7, 101, 54, 160

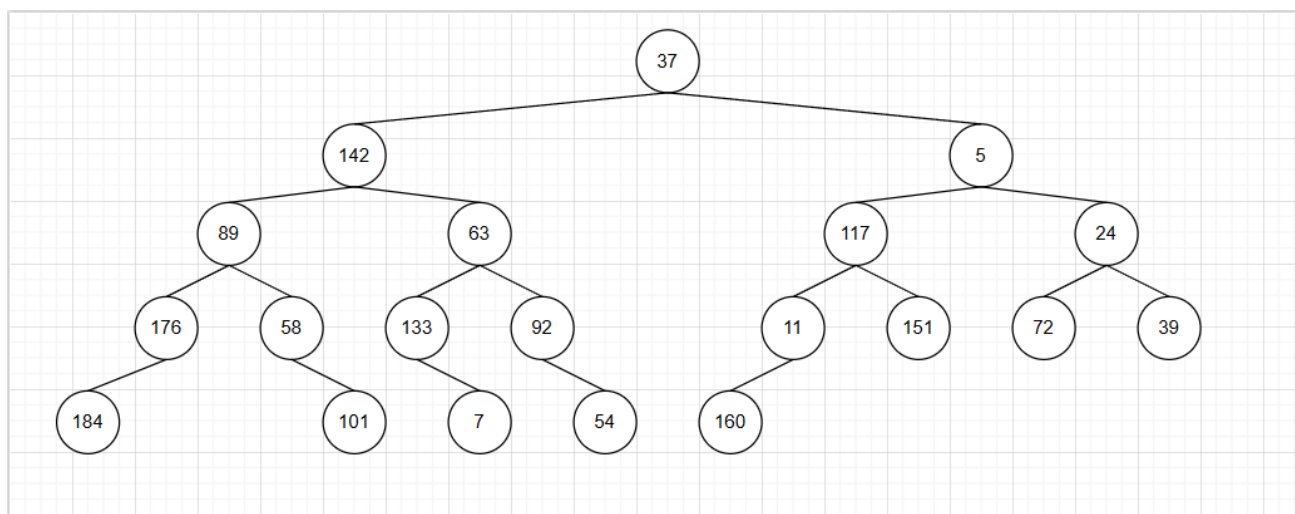
Task: For each tree type below, construct the tree using these integers, take a screenshot of the tree from your chosen tool, record the tool name/URL, and describe the insertion / heap-building procedure.

3.1 Binary Tree

Tool name / URL: <https://app.diagrams.net/?src=about>

Construction / insertion description: 在二元規則下依序排入

Screenshot of Binary Tree (paste below):

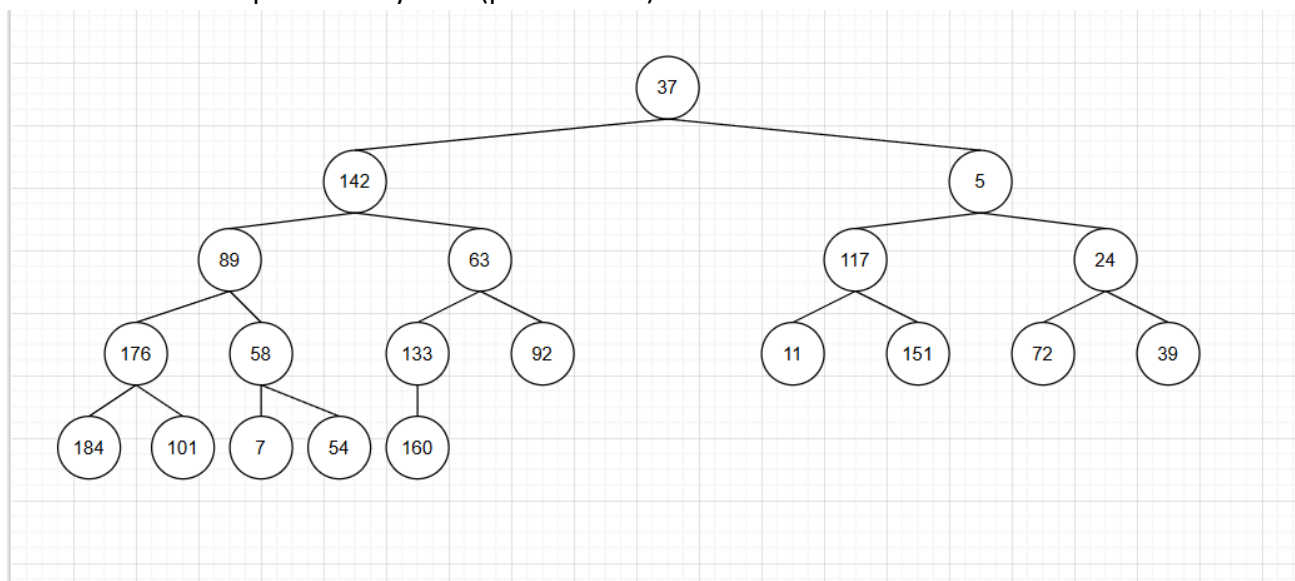


3.2 Complete Binary Tree

Tool name / URL: <https://app.diagrams.net/?src=about>

Construction / insertion description: 由左至右依序排入填滿

Screenshot of Complete Binary Tree (paste below):



3.3 Binary Search Tree (BST)

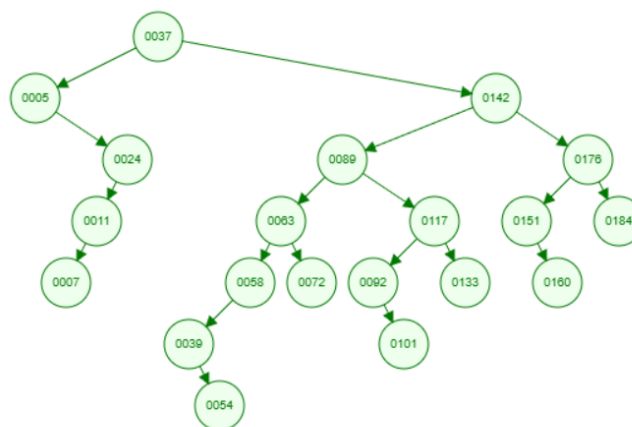
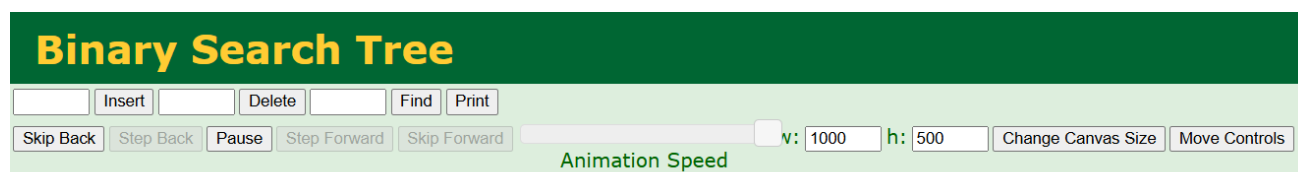
Student ID:

Student Name:

Tool name / URL: <https://www.cs.usfca.edu/~galles/visualization/BST.html>

Insertion rule (e.g., "insert in given order using BST rules"): 左小右大的規則排入

Screenshot of BST (paste below):



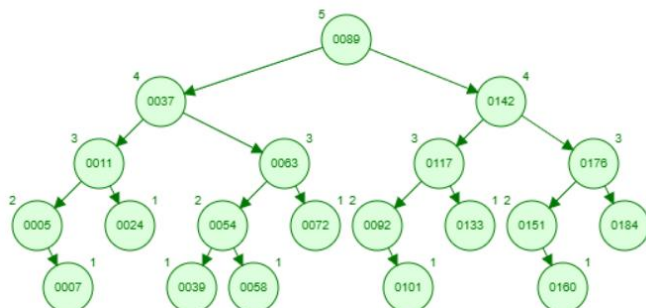
Animation Completed

3.4 AVL Tree

Tool name / URL: <https://www.cs.usfca.edu/~galles/visualization/AVLtree.html>

Insertion & balancing description: 根據 BST 的規則，但平衡每子樹的層數

Screenshot of AVL Tree (paste below):



3.5 Red-Black Tree

Student ID:

Student Name:

Tool name / URL: <https://www.cs.usfca.edu/~galles/visualization/RedBlack.html>

Insertion & balancing description: 以 BST 基礎加上紅色黑色進行排序

Screenshot of Red-Black Tree (paste below):

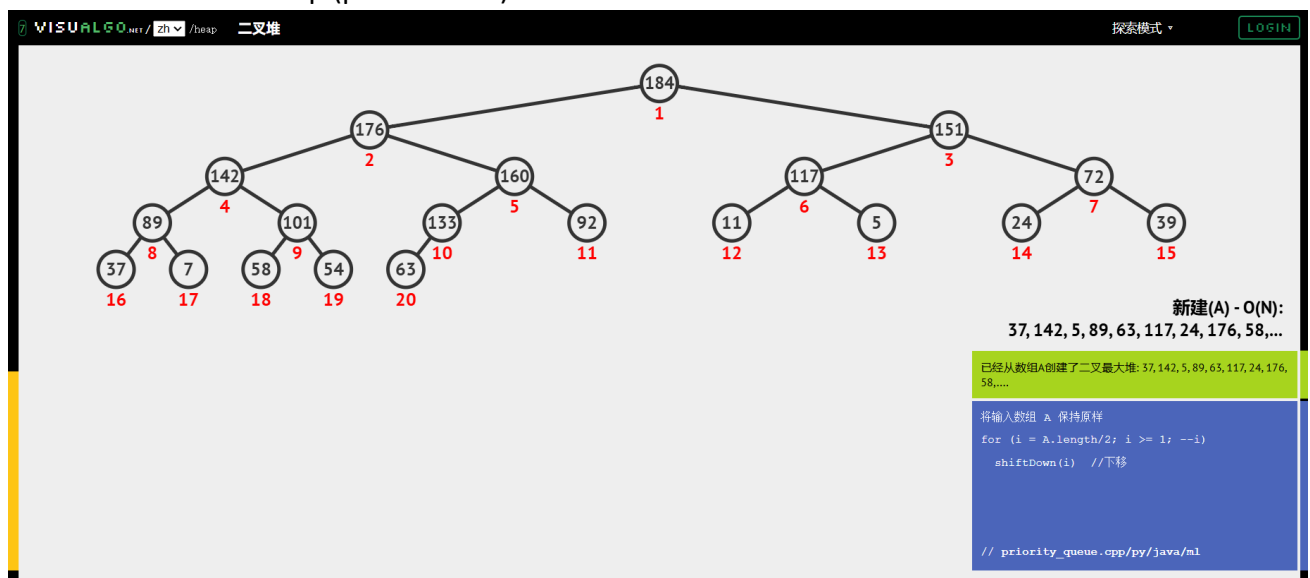


3.6 Max Heap

Tool name / URL: <https://visualgo.net/zh/heap>

Construction / heap-building description (e.g. heapify, insert-and-sift-up): 先依序排入後依據上下層大小更換位置

Screenshot of Max Heap (paste below):



3.7 Min Heap

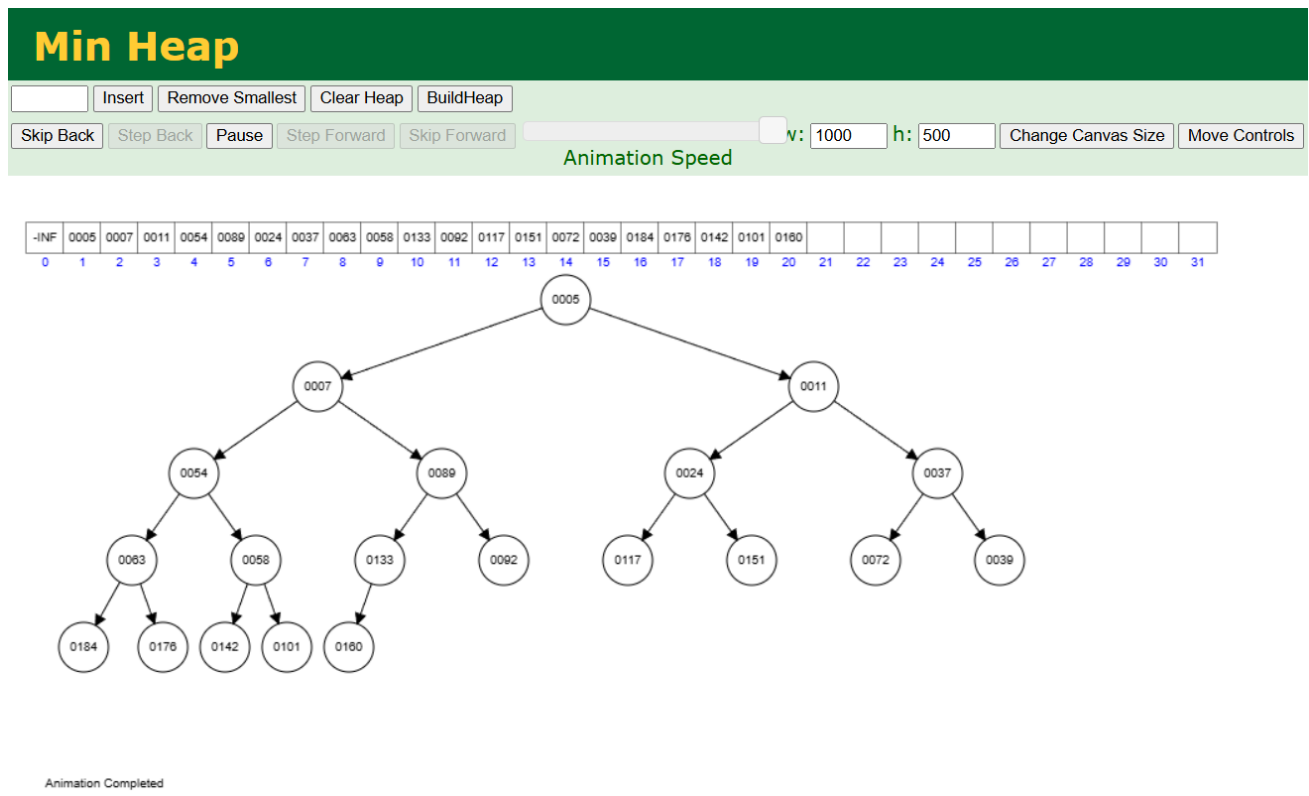
Tool name / URL: <https://www.cs.usfca.edu/~galles/visualization/Heap.html>

Construction / heap-building description: 先依序排入後依據上下層大小更換位置

Student ID:

Student Name:

Screenshot of Min Heap (paste below):



Section 4. Application Examples

Task: For each tree type, choose one application and explain why this tree is suitable.

Tree Type	Application Example (name / context)	Why this tree fits (properties that matter)
Binary Tree	編譯器	程式語言編譯器在讀 code 時，會把它變成一棵樹來分析語法
Complete Binary Tree	Max Heap 和 Min Heap	能保證把樹放進陣列時，中間絕對不會有空洞
Binary Search Tree	理論基礎與教學	容易歪掉變慢，現代軟體很少直接使用
AVL Tree	字典與拼字檢查	單字庫通常建立後很少改動，但需要極為頻繁的查詢，AVL 的嚴格平衡能提供最快的查詢速度
Red-Black Tree	一般用途的高效儲存	C++ 時用的 map 背後是紅黑樹
Max Heap	作業系統排程	如果有緊急任務插隊，Max Heap 能瞬間把它排到最前面處理
Min Heap	Google Maps (最短路徑)	使用 Min Heap 來不斷找出目前距離最近的路口

Section 5. Reflection on Tree Family and Performance (Optional but recommended)

Among BST, AVL, and Red-Black trees, which one would you pick for:

Student ID:

Student Name:

Mostly search (few updates)? Why?

AVL Tree，AVL Tree 是「嚴格平衡」，相比於紅黑樹，AVL Tree 通常壓得更扁、高度更低，搜尋的時間複雜度取決於樹的高度，因為 AVL 樹比較矮，所以在搜尋時需要比較的次數最少，讀取效率極高。

Frequent insertions and deletions? Why?

Red-Black Tree，紅黑樹的平衡條件比較寬鬆，當我們插入或刪除節點時，它不需要像 AVL Tree 那樣頻繁地進行旋轉來維持完美平衡。

If you must store these 20 integers for static search only (no updates), which structure or representation would you prefer (sorted array + binary search, BST, AVL, etc.)? Why?

Complete Binary Tree，形狀相對平衡搜尋快速，不需要大量的維護成本

Section 6. AI Usage Log (Required)

Task: Record every time you ask an AI assistant about this assignment.

Index	Date / Time	AI Service (ChatGPT, Gemini, etc.)	Your Full Prompt / Question
1	12/25 01:06	Gemini	有什麼推薦的工具嗎
2	12/25 01:12	Gemini	請問什麼是 general tree
3	12/25 9:50	Gemini	請問什麼是 binary tree
4	12/25 10:00	Gemini	請問什麼是 Complete Binary Tree
5	12/25 10:06	Gemini	請問什麼是 Binary Search Tree
6	12/25 10:12	Gemini	請問什麼是 AVL Tree
7	12/25 10:19	Gemini	請問什麼是 Red-Black Tree
8	12/25 10:27	Gemini	請問什麼是 max heap
9	12/25 10:33	Gemini	請問什麼是 min heap
10	12/25 11:38	Gemini	請問這些 tree 分別可以做什麼應用呢

You may extend this table as needed.